

넥스트칩 Apache6 SoM 보드 기반 어플리케이션 개발 프로젝트

넥스트칩 보드 활용 AI 객체 인식 시스템에서의
실시간 지정차로 단속 및 V2X 위험 경고 시스템

26.06.09 - 16 / B팀
정예찬 | 최명준 | 한선우 | 이은지



목차

| 01

프로젝트 개요 및 기획 배경

| 02

시스템 아키텍처

| 03

핵심 기능

| 04

트러블 슈팅

| 05

동작 영상

| 06

결론 및 고찰

1. 프로젝트 개요



프로젝트 주제

- ✓ **Nextchip Apache6 SoM 기반**
실시간 차선별 위반 단속 시스템
- ✓ **V2X 통신 규약을 활용한 차량 간**
위험 이벤트(급제동) 실시간 전파 및 시각화



활용 보드

- 🔧 **Model:** Nextchip Apache6 SoM
- ⚙️ **Performance:** 12 TOPS NPU
가속기 탑재
- 👁️ **Features:** 다채널 카메라 입력을
지원하는 고성능 ADAS Domain
Controller



개발 환경

- > **OS:** Ubuntu 20.04 LTS (Embedded)
- > **Language:** C
- 🧠 **AI Models:** YOLOv8 (객체), UFLD (차선)
- 📡 **Comm:** TCP Socket

1. 프로젝트 개요_기획배경

기존 지정차로단속의한계

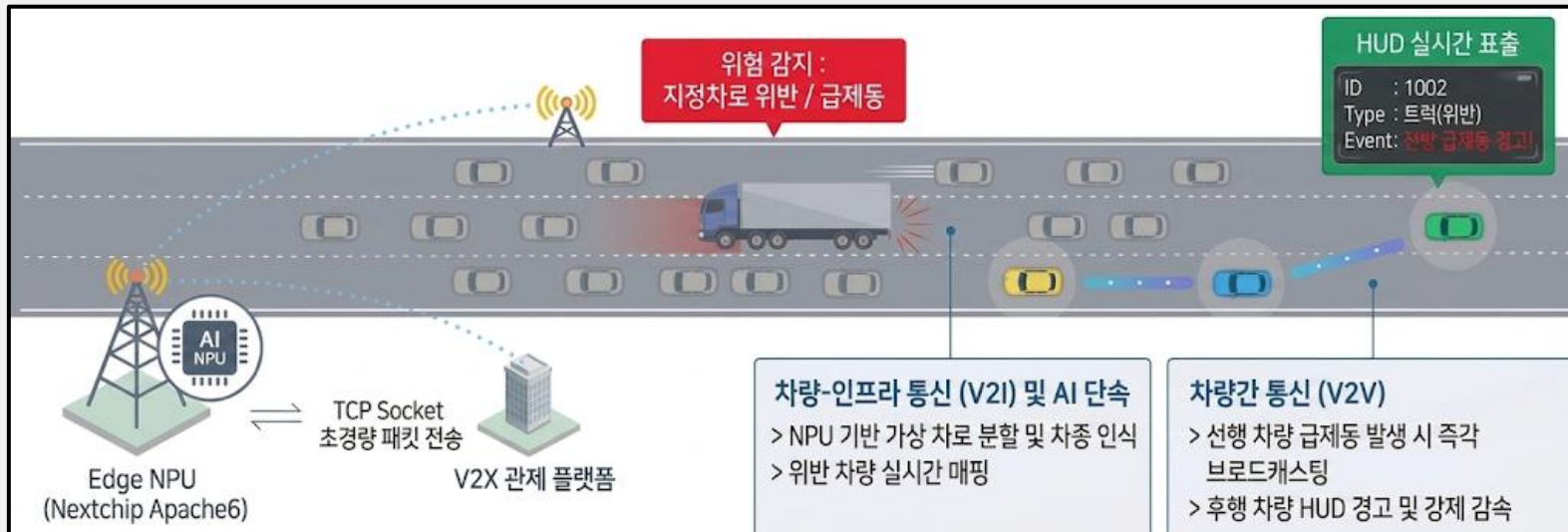
- 고정형 카메라 및 사후 신고 의존:
: 사각지대 통제 불가능, 시민들의 블랙박스 공익 신고에 의존
- 다중 차량 실시간 관제 어려움
: 다차선 위를 달리는 차량의 종류와 현재 차로를 즉각 매칭하는 시스템 부재



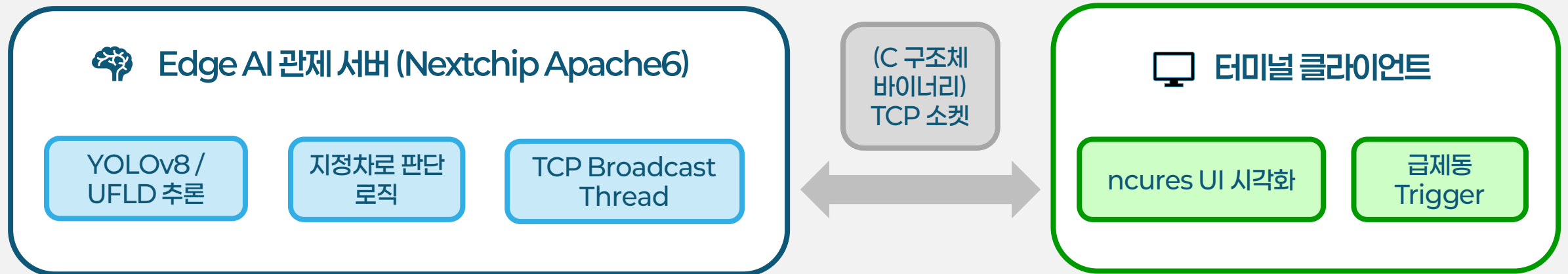
AI모델&V2X기반자동화단속

- AI 모델 기반 다차선-차종 실시간 판별
: Nextchip NPU만으로 차선과 진입 차량의 차종을 즉각 인식하여 위반 차량 자동 판별
- 사전 위험 예방(V2X Broadcasting)
: 위반 차량 적발, 전방 정체, 앞차량 급정거 등을 후방 차량 HUD에 실시간 경고 전달

프로젝트구상도:



2. 시스템 아키텍처



2. 시스템 아키텍처

지정차로 위반 감지

- ✓ **가상 차로 분할:**
1~4차로 실시간 매핑 및 차종(car/bus/tru/mot) 분류
- ✓ **위반 자동 판별:**
위반 차량 실시간 매핑 후 규격에 맞춰 구조체 생성 후 전달
- ✓ **전방 정체 판별:**
같은 차선에 차량이 3대 이상 인식 될 경우 정체로 판단

V2X 위험 이벤트 전파

- ✓ **이벤트 트리거:**
선행 차량 긴급 제동 발생 시([Spacebar]) 즉시 전파
- ✓ **차선 변경:**
[<-, ->] 좌우 방향키를 통해 차선 변경
- ✓ **능동형 제어:**
후행 차량 HUD 경고 표출 및 가상 속도 제어

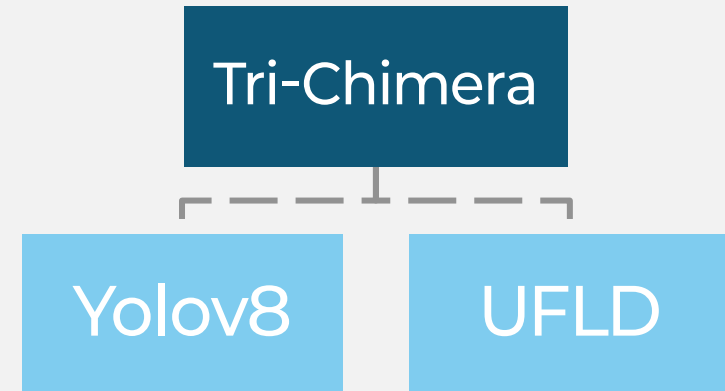
3.1 AI 객체 인식 및 차선 인식 로직 – 지정차로 위반 실시간 단속

1) 가상 카메라를 이용해 주행 영상 파일 실행



- 실제 하드웨어 카메라의 불안정성을 보완하기 위해 가상 카메라(Virtual Camera) 활용
- NPU 입력 규격
: 기존 RGB 형식에서 'RRR GGG BBB' Planar 변환

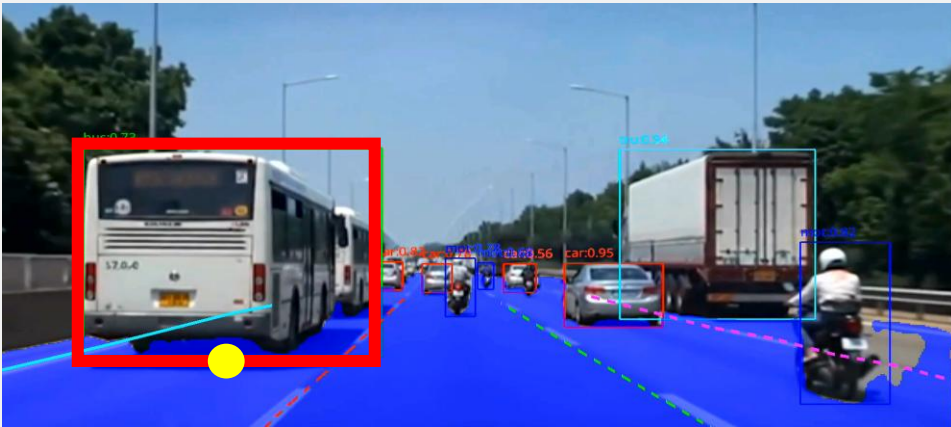
2) Tri-Chimera 모델 이용 (객체 인식 & 차선 인식)



- Yolov8 객체 검출
: Car, Bus, Truck, Mot 등 도로 객체 실시간 검출
- UFLD(Ultra Fast Lane Detection)
: 차선의 위치를 실시간으로 추정해 차선 경계 생성
: 좌/우 차선 정보를 이용해 현재 주행 차로 계산

3.1 AI 객체 인식 및 차선 인식 로직 – 지정차로 위반 실시간 단속

3) 객체 <-> 차로 Mapping



- YOLOv8으로 검출된 객체의 Bbox 검출
- 하단 중심점 추출
- UFLD가 검출한 차선 영역과 위치 비교
- 하단 중심점이 포함된 차선을 현재 주행 차로로 결정

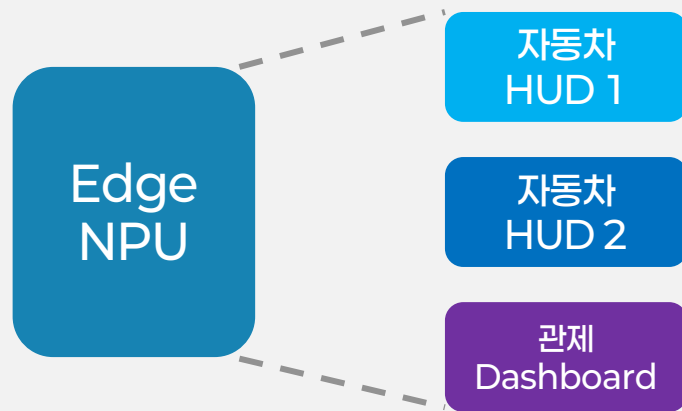
4) 지정차로 위반 판단

Class ID	Class	허용 차로
0	Car	2, 3차로
1	Bus	1차로
2	Truck	4차로
3	Motorcycle	진입 불가

- 차종별 허용 차로 규칙을 기반으로 주행 차로 비교
- 위반 차량 검출 시 event_code == 1 생성
- V2X 데이터 패킷 전송 준비
: lane, class_id, event_code, vehicle_id

3.2 TCP 소켓 통신 아키텍처

1) 1:N 다중 접속 브로드캐스팅 구조 (Multi-Threading)



- pthread 기반 독립 세션 관리
- Non-blocking 병렬 처리 :
특정 클라이언트 노드의 통신 지연이나 네트워크
이탈이 전체 V2X 경고 전파망에 영향을 주지 않도록
라우팅 차단 설계

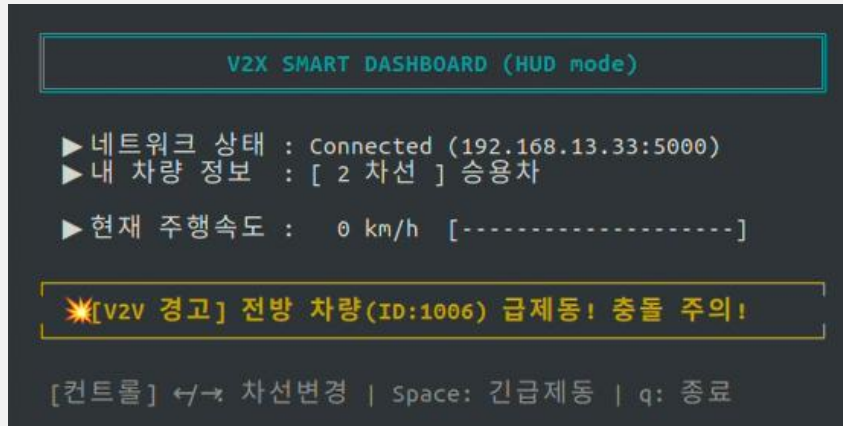
2) C 구조체 Payload 설계

```
typedef struct {  
    int32_t lane;           // 차선 (1, 2, 3...)  
    int32_t class_id;       // 0: 승용차, 1: 버스, 2: 트럭, 3: 오토바이  
    int32_t event_code;     // 0: 정상, 1: 지정차로 위반,  
                           // 2: 급제동 경고 3: 전방 도로 정체 감지  
    int32_t vehicle_id;  
} __attribute__((packed)) V2X_Data;
```

- 통신 딜레이를 최소화하기 위해 차선, 차종, 이벤트
코드, ID만 담은 C 구조체로 Payload 설계
- [__attribute__((packed))] 를 적용하여
불필요한 엔디안 변환 오버헤드 없이 패킷 크기 동기화

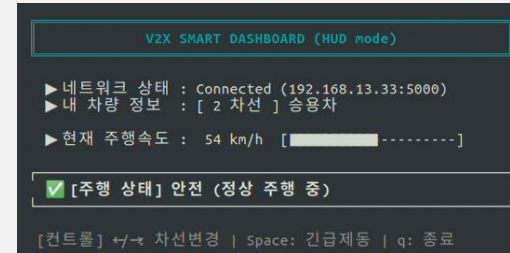
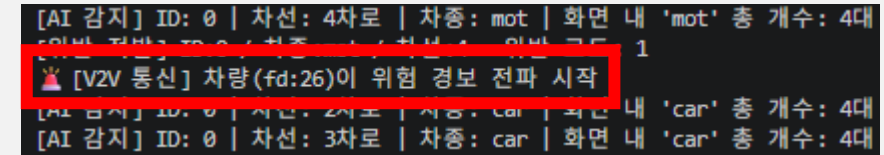
3.3 TCP 통신 – Event 양방향 V2V 제어

1) 클라이언트 선행 차량 긴급 이벤트 발생 전파 -> V2V



- 클라이언트(선행 차량)에서 운전자의 긴급 제동(Spacebar) 이벤트 발생 시, 즉각 상태를 event_code == 2 로 변환하여 서버로 역송신
- 해당 이벤트는 같은 차선 뒤에 차량에게 전파

2) 능동적인 제어 및 통합 관제와 동기화



- 수신 차량은 경고 출력에 그치지 않고 가상 주행 속도를 0km/h로 강제 제어
- 차선 수동 변경 기능 (<-,> 방향키를 통한 차선 이동)

4. 트러블슈팅

입력 소스 최적화를 통한 AI 모델 검출 신뢰성 확보

이슈(Issue)

소형객체 인식 불량(센서 왜곡)

- 영상을 카메라 센서를 통해 인식 시 빛 반사, 초점 흔들림 등의 노이즈가 대형 차량보다 크기가 작은 오토바이의 데이터를 집중적으로 훼손시킴. 이로 인해 확신도가 떨어져 임계값을 넘지 못하여 **오토바이 객체 미검출** 발생.



해결(Solution)

입력 소스 내장화를 통한 노이즈 근본적 차단

외부 카메라 센서 촬영 방식을 폐기하고, 타겟 보드 내에 주행 영상 파일을 넣는 Direct-File Feeding 구조로 전면 전환
렌즈 왜곡, 물리적 센서 노이즈를 제거하고 원본 픽셀 데이터 공급

```

lib
> demo
  cctv.mp4
  china.mp4
  finally.mp4
  fri1.mp4
  fri2.mp4
  fri3.mp4
  kakao1.mp4

if [ -z "$APP_PATH" ]; then
  APP_PATH=.
  fi

# 경로를 추가
export LD_LIBRARY_PATH=${APP_PATH}:${APP_PATH}/misc:${APP_PATH}/lib:${APP_PATH}/misc/lib:$LD_LIBRARY_PATH

LOOPBACK_CH1_VID=30

#영상 파일 경로
INPUT_FILE_PATH=${APP_PATH}/finally.mp4
insmod ${APP_PATH}/../modules/v4l2loopback_ko video_nr=$((LOOPBACK_CH1_VID+0)),${((LOOPBACK_CH1_VID+1))},${((LOOPBACK_CH1_VID+2))}
  
```

run_v4l2loopback.sh 파일에 경로 추가



정상 검출되는 결과 확인

```

[AI 감지] ID: 0 | 차선: 3차로 | 차종: mot | 화면 내 'mot' 총 개수: 3대
[위반 적발] ID: 0 / 차종: mot / 차선: 3 - 위반 코드: 1
[AI 감지] ID: 0 | 차선: 3차로 | 차종: mot | 화면 내 'mot' 총 개수: 3대
[위반 적발] ID: 0 / 차종: mot / 차선: 3 - 위반 코드: 1
[AI 감지] ID: 0 | 차선: 4차로 | 차종: mot | 화면 내 'mot' 총 개수: 3대
[위반 적발] ID: 0 / 차종: mot / 차선: 4 - 위반 코드: 1
  
```

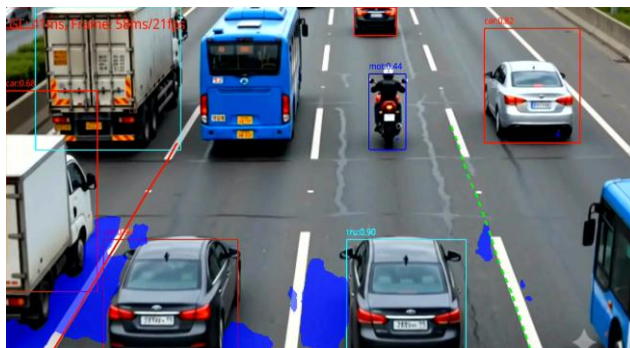
4. 트러블슈팅

AI UFLD, YOLO 모델 아키텍처 분석 및 전략적 한계 설정

이슈(Issue)

고정 CCTV 시점에서의 AI 인식률 급감

- 도로 관제 구현을 위해 고정형 CCTV 시점을 도입했으나 인식률 급감
- UFLD 내부 파라미터 분석 결과, 모델이 대시캠 시점에 최적화 되어 있어, CCTV 영상과 데이터 도메인 불일치가 발생



```
row_anchor_num = 18 ; number of row anchors  
row_anchor_min = 0.2  
row_anchor_max = 1
```

UFLD ini 파일 일부

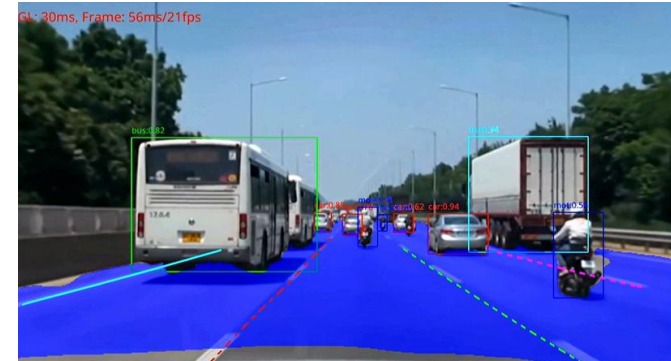
UFLD ini 설정 기반 원인 분석

- row_anchor_min = 0.2 → 상단 20% 미스캔 → CCTV 차선 탐색 불가
- row_anchor_num = 18 → CCTV 양각·렌즈 왜곡으로 X,Y 교차점 학습 범위 초과

해결(Solution)

최적 각도 도출 및 시나리오 수정

- 프로젝트 보드 한계 상 AI 재학습을 할 수 없어, 수차례 테스트를 통해 최적의 고도와 양각의 영상을 산출
- 기존 고정형 CCTV 기점이 아닌 드론과 비슷한 관점의 관제시스템으로 시나리오를 수정하였고 객체의 인식률을 높임.



4. 트러블슈팅

AI 모델의 인덱스와 V2X 프로토콜 간의 불일치 해결

이슈(Issue)

AI Raw Data와 통신 규격 간의 충돌 위험

- YOLOv8 모델이 출력하는 인덱스 체계와 사전에 정의한 V2X 통신 프로토콜 명세와 달랐던 문제

YOLO cls: 2 (Car), 5 (Bus), 0 (Person)

✗ 그대로 전송 시 시스템 붕괴

V2X Protocol: 0 (Car), 1 (Bus), 2 (Truck)

해결(Solution)

데이터 매핑 레이어 구축

- server-client 직전에 독립적인 데이터 매핑/필터링 레이어를 구현
- 프로토콜 규격에 맞게 재매핑하고, 관제 대상이 아닌 데이터는 필터링



4. 트러블슈팅

네트워크 불안정성으로 인한 패킷 분할 수신 및 쓰레기 값 처리 문제

이슈(Issue)

데이터 파편화 현상 발생

- 기존 'recv()' 호출 방식에서 네트워크 지연 발생 시 16바이트 구조체가 10바이트, 6바이트로 쪼개져서 도착하는 현상 발생
- 구조체 내부의 데이터가 밀리면서 16777216과 같은 쓰레기 값이 출력

```
int bytes = recv(sock, &data, sizeof(data), 0);
if (bytes > 0) {
    // 데이터가 덜 왔는데도 구조체 처리를 강행함
    process_v2x_data(&data);
}
```

[1781159277] 📡 데이터 수신 -> 차선:50331648, 위반코드:16777216
[1781159277] 📡 데이터 수신 -> 차선:16777216, 위반코드:16777216
[1781159277] 📡 데이터 수신 -> 차선:33554432, 위반코드:0

해결(Solution)

고정 길이 누적 수신 로직 설계

- TCP 통신이 '바이트 스트림(Byte Stream)'이라는 본질에 입각하여 전체 구조체 사이즈에 도달할 때까지 남은 바이트 크기 만큼 포인터 오프셋을 이동시키며 누적 수신 하도록 수신 모듈 수정

```
int total_received = 0;
int data_size = sizeof(V2X_Data);
char *ptr = (char *)&net_data;

while (1) {
    // total_received가 data_size가 될 때까지 계속 읽음
    int n = recv(sock_fd, ptr + total_received, data_size - total_received, 0);

    if (n <= 0) break; // 서버 접속 종료

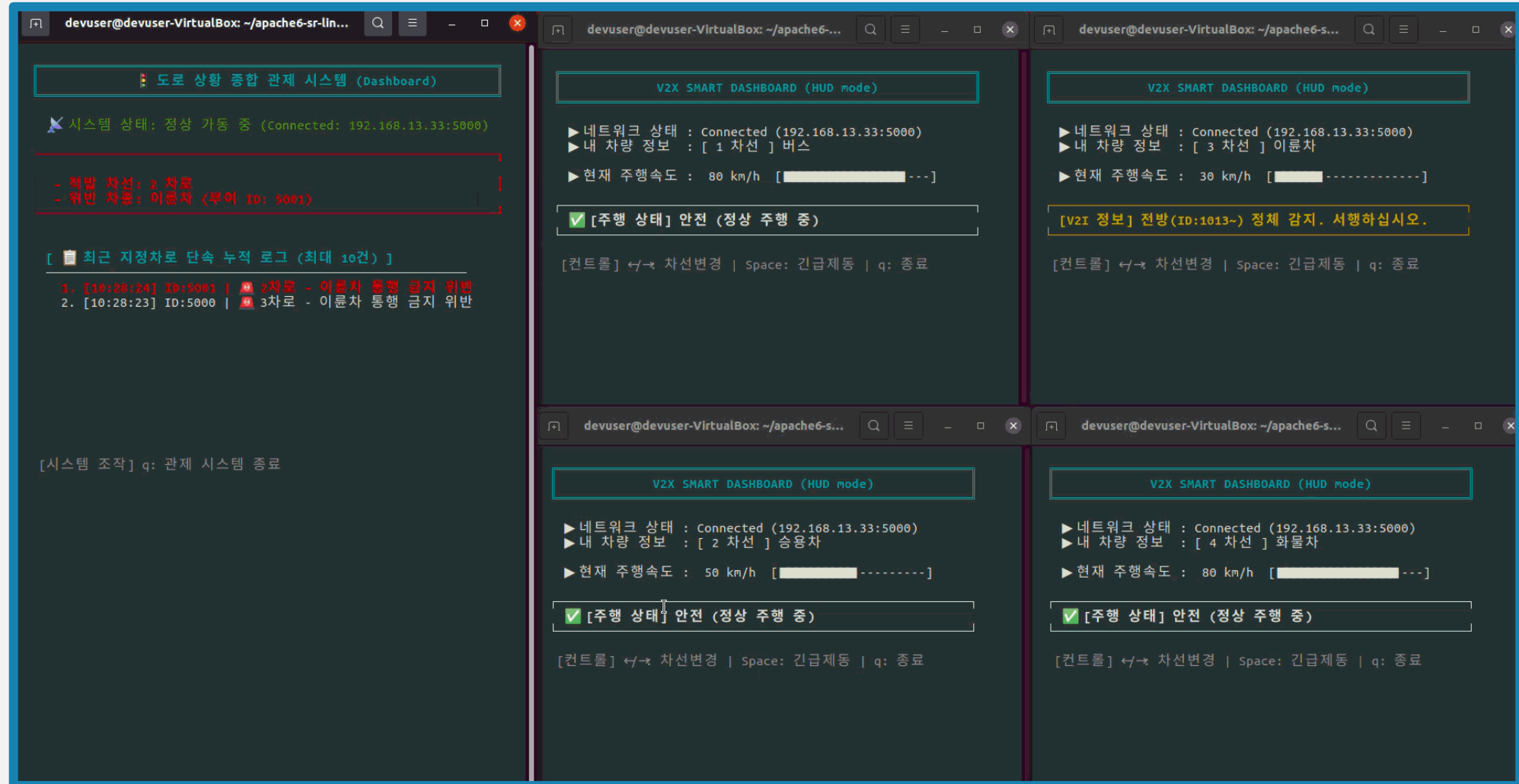
    total_received += n;
}
```

지정차로 위반 차량 단속



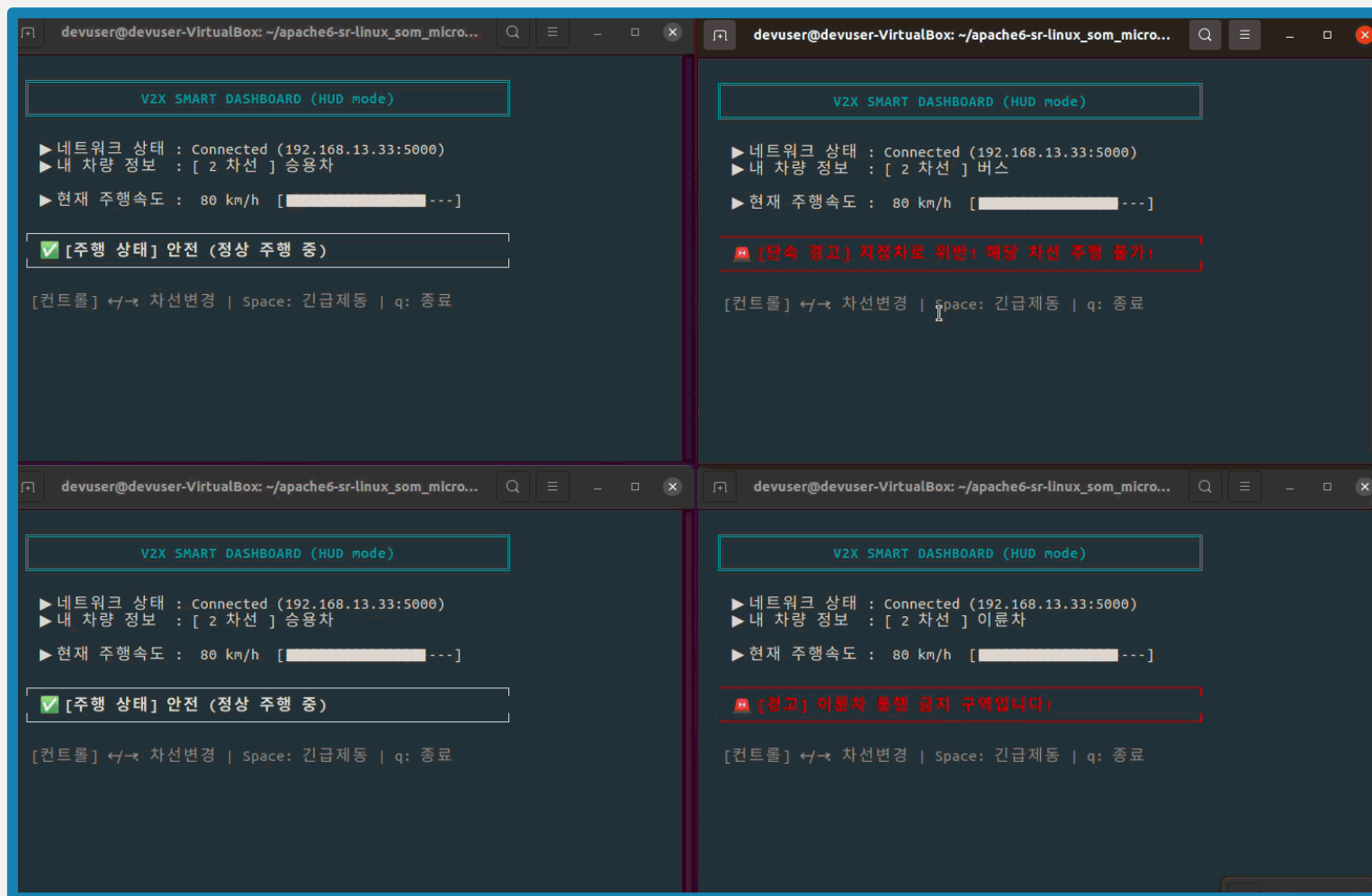
5. 동작 영상

터미널 전체 동작 영상



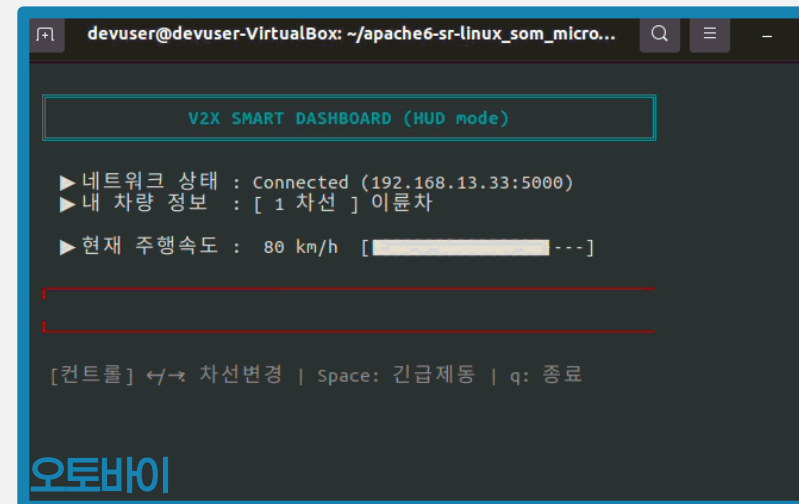
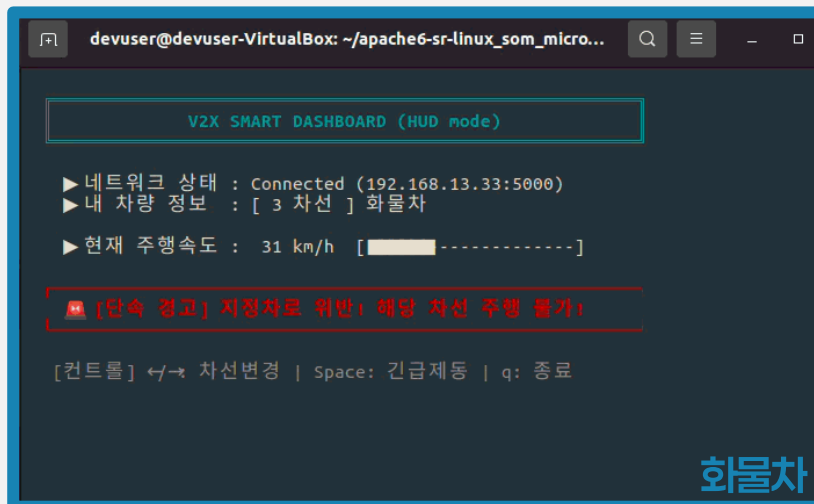
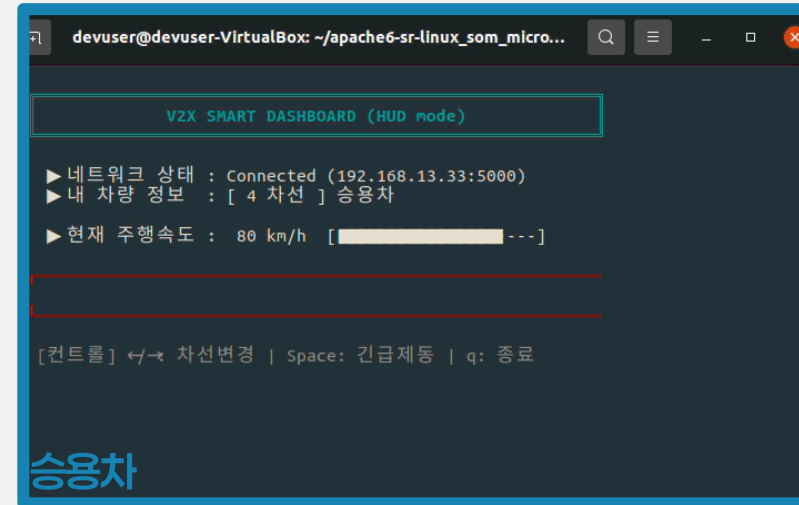
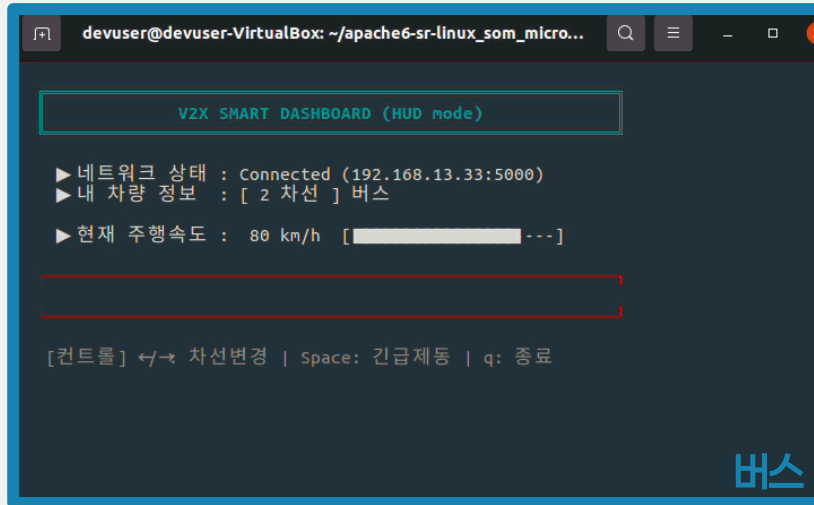
5. 동작 영상

V2V전방차량 급제동 전파



5. 동작 영상

차량별 차선 이동 및 차선 위반 경고



6. 결론 및 향후 과제



프로젝트 성과

- ✓ SDV 구현:
하드웨어 제약을 소프트웨어 로직으로 극복한 ADAS 어플리케이션 개발
- ✓ TCP 소켓 통신 활용 저지연 통신 환경 구축
- ✓ 지정차로 위반 단속 관제 동작 검증 완료:
지정차로 위반 감지 및 V2X 긴급 제동 이벤트, 전방 정체 알림 동작 확인



향후 발전 과제

- ✓ Fine-tuning 수행:
CCTV와 같은 고정형 카메라 영상을 추가 수집하여, 다양한 카메라 환경에서도 안정적으로 동작하는 차선 검출 모델로 개선
- ✓ 통신 고도화:
현재의 TCP 소켓 통신을 실제 무선 환경으로 전환

Conclusion

6. 고찰 및 Q&A



정예찬

**TCP통신로직
구현및코드통합**

시스템 간의 통신에서 가장 중요한 것은 AI의 추론 결과를 통신 규격에 맞게 경량화하고 매칭하는 데이터 정합성임을 체득했습니다. C구조체와 TCP 소켓 통신 로직을 직접 개발하며, 데이터 파이프라인의 견고함이 곧 전체 서비스의 성능을 결정함을 배웠습니다.

또한 시스템 간의 결합에서 발생하는 문제는 명확한 인터페이스 설계와 팀원과의 긴밀한 소통으로 극복할 수 있음을 배웠습니다.



최명준

**TCP통신및
테스트영상 최적화**

이번 프로젝트를 통해 실제 임베디드 하드웨어인 Apache6 SoM에서 구동되는 ADAS 시스템이 어떤 단계별 과정을 거쳐 사용자에게 경고 메시지를 전달하는지, 그 기술적 흐름을 전체적으로 조망할 수 있었습니다.

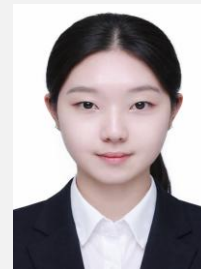


한선우

**EdgeAI
시스템최적화**

타겟 보드 세팅부터 데모 검증까지 디버깅과 트러블슈팅을 거치며, 임베디드 하드웨어 시스템의 인풋부터 아웃풋까지의 디지털 설계 구조를 이해하는 계기가 되었습니다.

수많은 에러와 트러블슈팅 과정을 팀원과의 긴밀한 소통으로 극복하며, 안정적인 시스템 완성을 위한 협업의 가치를 체득했습니다.



이은지

**EdgeAI
로직구현**

Tri-Chimera를 기반으로 검출된 데이터를 활용하는 과정에서 어떤 정보를 추출하고 어떤 형태로 가공하여 전송할지에 대한 고민이 많았습니다.

객체 검출과 차선 검출 결과를 분석하여 핵심 정보를 정의하는 과정을 통해 AI 추론 결과를 실제 서비스에 활용하기 위한 데이터 설계의 중요성을 배울 수 있었습니다.